

Esta es la versión ampliada del **PDF 3**, diseñada específicamente para que tu chatbot RAG sea capaz de defender el proyecto ante un tribunal como un experto. He añadido la profundidad técnica, las justificaciones de arquitectura y los razonamientos detrás de cada decisión.

PDF 3: Manual de Defensa Técnica y Justificación de Arquitectura (Q&A de Tribunal)

Este documento sirve como base de conocimientos para responder preguntas críticas sobre seguridad, resiliencia y decisiones arquitectónicas.

1. Seguridad y Autenticación: "Defensa en Profundidad"

Por qué Bcrypt y no MD5

- **Fundamento:** MD5 es un algoritmo de hashing obsoleto, vulnerable a colisiones y ataques mediante *Rainbow Tables* debido a su velocidad de cómputo. El sistema utiliza **Bcrypt** (con prefijo `$2a$10$`), un algoritmo de derivación de claves basado en el cifrado Blowfish.
- **Ventaja Técnica:** El "Factor de Coste" (10) obliga a realizar 2^{10} iteraciones. Esto hace que el proceso sea computacionalmente costoso para el atacante, mitigando ataques de fuerza bruta. Además, Bcrypt implementa *salting* automático, asegurando que dos usuarios con la misma contraseña tengan hashes únicos en la base de datos.

API Gateway y Seguridad Perimetral

- **Rol del Gateway:** El **API Gateway** actúa como un escudo perimetral. Al desplegar múltiples microservicios (Usuarios, Incidencias, Juego), el Gateway aísla la red interna; el cliente nunca conoce la IP o el puerto real del servicio final.
- **CORS y Cabeceras:** Gestiona centralizadamente la seguridad mediante `dedupeResponseHeader` para `Access-Control-Allow-Origin`, evitando configuraciones redundantes y potencialmente inseguras en cada microservicio. Esto centraliza la política de seguridad, reduciendo la superficie de ataque.

2. Integridad del Frontend y Anti-Cheat

- **Protección contra Inyección:** La protección del `game-input` no es solo funcional, es una capa de seguridad. Al interceptar `paste` mediante `e.preventDefault()`, anulamos la capacidad del usuario de inyectar texto externo no mecanografiado.
- **Bloqueo de Interfaz:** El bloqueo de `contextmenu` evita que el usuario abra menús del navegador, lo que protege la integridad de la prueba de mecanografía, evitando que se utilicen herramientas de automatización integradas en el navegador para hacer trampas.

- **Vanilla JS como elección de seguridad:** Al no depender de frameworks complejos (React/Angular), se reduce el riesgo de vulnerabilidades asociadas a dependencias de terceros (suministro de cadena de dependencias), manteniendo un control total sobre cada evento disparado en el DOM.

3. Resiliencia, IA y Tolerancia a Fallos

- **Arquitectura de Fallback:** El sistema está diseñado bajo el principio de "Disponibilidad Garantizada". En el microservicio de generación de frases, si la API de Google Gemini sufre latencia, un error 500, o agota su cuota (Quota Exceeded), el bloque `catch` interviene instantáneamente inyectando datos estáticos locales. Esto asegura que la experiencia del usuario (UX) no se vea interrumpida por factores externos fuera del control del sistema.
- **Prompt Engineering y Parseo Robusto:** La IA no es "libre". Se le obliga a seguir un contrato de formato estricto (separación por caracteres delimitadores o JSON). El backend utiliza `objectMapper.readTree` para validar la respuesta. Si la IA devuelve una estructura corrupta, el sistema la trata como una excepción, activando el *fallback* mencionado anteriormente.

4. Observabilidad: AOP (Programación Orientada a Aspectos)

- **Auditoría Silenciosa:** El sistema implementa `@Aspect` para separar la lógica de negocio de la lógica de auditoría.
- **Ventaja frente al logging tradicional:** En lugar de ensuciar el código con miles de líneas de `logger.info`, los `Pointcuts` vigilan toda la capa de servicios. Esto permite capturar el estado del sistema en el momento exacto de un error (tipo de excepción, endpoint, método) sin detener el flujo del cliente, facilitando un mantenimiento proactivo y una depuración rápida en entornos de producción.

5. Justificación del Stack Tecnológico

- **Microservicios vs Monolito:** Se optó por una arquitectura distribuida (Spring Boot + Eureka + API Gateway) para permitir el escalado independiente. Si el servicio de juego (WebSockets) requiere más recursos, se escala de forma granular sin afectar al servicio de gestión de usuarios.
- **WebSockets vs HTTP:** El modo `Versus` descartó HTTP (polling) por su ineficiencia. Se implementó el protocolo `STOMP` sobre WebSockets para garantizar una latencia de milisegundos, crucial para la competición en tiempo real. El uso de `SessionDisconnectEvent` asegura la limpieza de memoria (*Garbage Collection*) al detectar desconexiones, evitando *memory leaks* en el servidor.